

Table of contents

1. bin2cell_AB01.ipynb / bin2cell_AB02.ipynb — Visium HD bin-to-cell segmentation
2. filter_blood_cells.ipynb — cropped out blood cells spatially from visium HD outputs
3. combined_samples.ipynb — Concatenate both samples (AB01, AB02), QC, normalization, batch correction, and clustering
4. reclustering.ipynb — cell type annotation, SEC cell identification, and subset reclustering
5. tissues_one_canvas.ipynb — multi-tissue coordinate stitching onto a shared canvas
6. plot_with_grid.py — coordinate grid plotting utilities for spatial region identification - this is just a utils script
7. trajectory_analysis_original.ipynb — exploratory pdgfra/tdtomato trajectory preprocessing - this is not final, just drafting
8. pdgfra_analysis.ipynb — Trajectory analysis for pdgfra+ cells
9. tdtomato_analysis.ipynb — Trajectory analysis for tdTomato+ cell
10. trajectory.py — HPC full trajectory pipeline (pdgfra+ and tdtomato+) - this is just exploratory script when I was playing with Cellrank
11. trajectory_pdgfra_only.py — HPC trajectory pipeline (pdgfra+ only) - just exploratory script
12. resume_lineage_drivers.py — resume lineage driver computation from checkpoint - just exploratory script
13. run_trajectory.sh — SLURM job submission script - this is because I used to run these bigger scripts on Columbia's HPC

1. bin2cell_AB01.ipynb / bin2cell_AB02.ipynb

Visium HD bin-to-cell segmentation using bin2cell.

files:

Note: change relative paths for these files. * **input files::** * AB01: *
/content/drive/MyDrive/mansfield-lab/visium/cell_segmentation/AB01/
* square_002um/ * AB01.tiff * AB02: * /content/drive/MyDrive/mansfield-lab/visium/cell_segmentat
* source 2um binned data: square_002um/ * source H&E image: AB02.tiff *
output files: * AB01: * AB01_b2c.h5ad - /Users/rikac/mansfield_lab/spring_26/datasets/AB01_b2c.h5
* Other outputs @ /Users/rikac/mansfield_lab/spring_26/cell_segmentation/AB01/
* AB01_gex.npz * AB01_gex.tiff * AB01_he.npz * AB01_he_scaled.tiff *
AB02: * AB02_b2c.h5ad - /Users/rikac/mansfield_lab/spring_26/datasets/AB02_b2c.h5ad
* Other outputs @ /Users/rikac/mansfield_lab/spring_26/cell_segmentation/AB02/
* AB02_gex.npz * AB02_gex.tiff * AB02_he.npz * AB02_he_scaled.tiff

parameters:

parameter	AB01	AB02
mpp (microns per pixel)	0.5029	0.5027
prob_thresh H&E	0.0003	0.0003
prob_thresh GEX	0.05	0.05
nms_thresh H&E	0.3	0.3
nms_thresh GEX	0.5	0.5
volume_ratio	4	4
sigma (GEX smoothing)	5	5

results summary:

step	AB01	AB02
input bins	4,981,472	3,753,489
bins after filtering	2,525,920	2,204,889
H&E nuclei detected	111,592 objects → 1,040,976 bins	67,859 objects → 755,311 bins
bins after label expansion	1,501,826	1,053,358
GEX cells detected	14,079 objects → 526,161 bins	19,501 objects → 572,930 bins
secondary labels salvaged	243	550
final segmented bins	1,507,068	1,066,941
final cells	111,577	68,183
primary label source	111,334 (99.8%)	67,633 (99.2%)
secondary label source	243 (0.2%)	550 (0.8%)

process:

1. load Visium HD data

1. data is loaded from the 2 μ m bin output of Space Ranger using `b2c.read_visium()`
2. gene names are made unique with `var_names_make_unique()`
3. initial AnnData object contains bins x genes

2. data cleaning

1. filter genes expressed in fewer than 3 bins: `sc.pp.filter_genes(adata, min_cells=3)`
2. filter bins with zero counts: `sc.pp.filter_cells(adata, min_counts=1)`

3. prepare H&E image for segmentation

1. the source H&E image is rescaled to a standard resolution using `b2c.scaled_he_image()`
2. `mpp` (microns per pixel) is derived from the image metadata
3. the function crops and scales the image, and stores new spatial coordinates under `spatial_cropped_150_buffer`

4. save raw counts

1. before any further processing, raw counts are saved as a layer:
`adata.layers['raw_counts'] = adata.X.copy()`

[!note] destriping `b2c.destripe()` was explored to correct the characteristic striped artifact in Visium HD 2 μm bins (caused by row/column width variability). it is currently `==not applied==` to either sample — the step is commented out in both notebooks.

5. H&E-based nuclei segmentation (primary)

1. StarDist model `2D_versatile_he` is run on the scaled H&E image via `b2c.stardist()`
2. parameters: `prob_thresh=0.0003` (lowered from default 0.692 to capture more nuclei), `nms_thresh=0.3`
3. labels are loaded back into the AnnData with `b2c.insert_labels()`, stored as `labels_he`
4. bins assigned to a nucleus receive a non-zero integer label; unassigned bins are 0

6. expand nuclei to cell boundaries

1. H&E StarDist identifies nuclei only, not full cells
2. `b2c.expand_labels()` with `algorithm="volume_ratio"` and `volume_ratio=4` expands each nucleus label outward
 1. `volume_ratio=4` is the default and represents the expected ratio of cell volume to nuclear volume
 2. bins equidistant between two nuclei are resolved by comparing gene expression profiles in PCA space
3. expanded labels stored as `labels_he_expanded`

7. GEX-based cell segmentation (secondary)

1. a pseudo-fluorescence image is created from `n_counts` per bin using `b2c.grid_image()` with `sigma=5` Gaussian smoothing
2. StarDist model `2D_versatile_fluo` is run on this image via `b2c.stardist()`
3. this model detects cells directly (not just nuclei), so no label expansion step is needed

4. labels are loaded with `b2c.insert_labels()`, stored as `labels_gex`
5. this step is used to capture cells in denser tissue regions where H&E segmentation may miss cells

8. combine H&E and GEX labels

1. `b2c.salvage_secondary_labels()` merges the two label sets
 1. primary: `labels_he_expanded` (H&E-based)
 2. secondary: `labels_gex` (GEX-based)
2. GEX-based labels are only kept for regions not already covered by H&E labels
3. combined labels stored as `labels_joint`; source tracked in `labels_joint_source` ("primary" or "secondary")

9. aggregate bins into cells

1. `b2c.bin_to_cell()` groups all bins sharing the same `labels_joint` value
2. the resulting `AnnData` (`cdata`) has one observation per cell rather than per bin
3. spatial coordinates are aggregated for both `spatial` and `spatial_cropped_150_buffer` keys
4. `cdata.obs` contains: `object_id`, `bin_count`, `array_row`, `array_col`, `labels_joint_source`

10. save output

1. raw counts are copied to `cdata.layers['raw_counts']` before saving
2. `cdata.write_h5ad()` saves the final cell-level `AnnData` with `compression='gzip'`
3. AB01 output: `AB01_b2c.h5ad`
4. AB02 output: `AB02_b2c.h5ad`

2. filter__blood__cells.ipynb

filters contaminated blood cells in the segmented cells spatially.

files:

- **input files:**
 - `AB01_b2c.h5ad` — bin2cell output for slide AB01; set path via `AB01_PATH` in the notebook
 - `AB02_b2c.h5ad` — bin2cell output for slide AB02; set path via `AB02_PATH` in the notebook
- **output files:**
 - `AB01_filtered.h5ad` — cropped and filtered `AnnData` for AB01; set path via `AB01_OUT` in the notebook

- `AB02_filtered.h5ad` — cropped and filtered AnnData for AB02; set path via `AB02_OUT` in the notebook
- `<sample>_matrix/` — 10x-format sparse count matrix (barcodes, features, `matrix.mtx`); set directory via `MTX_OUT` in the notebook
- `<sample>_boundaries.geojson` — GeoJSON file of cell polygon boundaries; set path via `GEOJSON_OUT` in the notebook

parameters:

parameter	value (AB01)	value (AB02)	description
<code>array_col</code> crop range	0–2700	0–2800	x-axis pixel coordinate bounds for tissue region
<code>array_row</code> crop range	575–2875	600–2955	y-axis pixel coordinate bounds for tissue region
min cells per gene	10	10	minimum number of cells a gene must appear in to be retained

process:

1. load bin2cell output

1. read `AB01_b2c.h5ad` and `AB02_b2c.h5ad` using `sc.read_h5ad()`
2. inspect spatial coordinate ranges in `adata.obs['array_col']` and `adata.obs['array_row']` to identify tissue extent

2. crop to tissue region

1. apply boolean masks on `array_col` and `array_row` to select cells within tissue bounds
2. subset each AnnData with the mask to retain only in-tissue cells
3. verify cell counts before and after cropping

3. filter blood cells and low-coverage genes

1. remove known blood cell cluster(s) identified from prior clustering (by `leiden` label or marker genes)
2. filter genes expressed in fewer than the minimum number of cells using `sc.pp.filter_genes()`

4. export filtered h5ad

1. write each filtered AnnData to disk with `adata.write_h5ad(AB01_OUT)` and `adata.write_h5ad(AB02_OUT)`

5. export 10x mtx format

1. export sparse count matrix using `sc.readwrite` or `scipy.io.mmwrite()` to produce `matrix.mtx`, `barcodes.tsv`, and `features.tsv`

6. export GeoJSON polygon boundaries

1. retrieve cell segmentation label images from `adata.uns` or from the StarDist output
2. use `skimage.measure.regionprops` to extract bounding boxes and `skimage.measure.find_contours` to trace polygon boundaries per cell
3. call `export_polygon_geojson()` to serialize polygon coordinates per cell as a GeoJSON FeatureCollection
4. write to `<sample>_boundaries.geojson`

7. save as HDF5-compatible SpatialData format

1. package filtered count matrix and spatial coordinates into an HDF5-compatible structure for downstream use with the SpatialData framework

3. combined_samples.ipynb

This script combines both samples AB01 and AB02 and saved it to `h5ad` as the output file. It first constructs SpatialData objects for each sample, concatenates them, performs QC filtering, normalization, Harmony batch correction, Leiden clustering at multiple resolutions, and computes DEGs. runs on Google Colab.

files:

- **input files:**
 - `AB01_filtered.h5ad` and `AB02_filtered.h5ad` — filtered AnnData files from step 2; set paths via `AB01_PATH` and `AB02_PATH`
 - `AB01_boundaries.geojson` and `AB02_boundaries.geojson` — GeoJSON polygon boundaries; set paths via `AB01_GEO` and `AB02_GEO`
- **output files:**
 - `concatenated_sdata/` — zarr-backed SpatialData object containing both samples; set path via `SDATA_OUT`
 - `combined_samples.h5ad` — concatenated and processed AnnData; set path via `ADATA_OUT`
 - `degs_leiden_0.6.csv` and `degs_leiden_0.7.csv` — DEG tables per Leiden resolution; set directory via `DEG_OUT`

parameters:

parameter	value	description
min total counts (cell filter)	53	minimum total UMI counts per cell
max total counts (cell filter)	22025	maximum total UMI counts per cell (doublet/artifact cutoff)
min cells per gene	50	minimum number of cells a gene must appear in
normalization target sum	1e4	target total counts per cell after normalize_total
Harmony key	"sample"	obs column used for batch correction
Leiden resolution 1	0.6	coarser clustering resolution
Leiden resolution 2	0.7	finer clustering resolution
n PCA components	50	number of principal components before Harmony
n neighbors	15	number of neighbors for UMAP/Leiden graph

process:

1. create zarr SpatialData objects

1. for each sample, call `create_zarr()` with the filtered h5ad and GeoJSON boundaries to produce a zarr-backed SpatialData object
2. zarr objects encode count matrices, spatial coordinates, and cell polygon shapes

2. concatenate samples

1. call `spd.concatenate([sdata_AB01, sdata_AB02])` to merge both SpatialData objects into `concatenated_sdata`
2. add a `sample` column to `.obs` to track slide of origin

3. quality control

1. flag mitochondrial genes with `adata.var_names.str.startswith("mt-")` and compute `pct_counts_mt`
2. filter cells: retain only cells with total counts between `min_counts` and `max_counts`
3. filter genes: retain only genes expressed in at least `min_cells` cells using `sc.pp.filter_genes()`
4. inspect QC distributions via violin and scatter plots

4. normalization

1. normalize total counts to 10,000 per cell with `sc.pp.normalize_total(target_sum=1e4)`
2. apply log1p transformation with `sc.pp.log1p()`

3. store the result in `adata.layers['norm_log']` for downstream use

5. feature selection and PCA

1. identify highly variable genes with `sc.pp.highly_variable_genes()`
2. compute PCA on the HVG subset with `sc.pp.pca(n_comps=50)`

6. Harmony batch correction

1. run Harmony using `sce.pp.harmony_integrate(adata, key="sample")` to correct for slide-level batch effects
2. store corrected embeddings in `adata.obsm['X_pca_harmony']`

7. neighborhood graph and UMAP

1. compute the neighborhood graph with `sc.pp.neighbors(use_rep='X_pca_harmony', n_neighbors=15)`
2. compute UMAP with `sc.tl.umap()`

8. Leiden clustering

1. run `sc.tl.leiden(resolution=0.6)` and store as `leiden_harmony_0.6`
2. run `sc.tl.leiden(resolution=0.7)` and store as `leiden_harmony_0.7`
3. visualize clusters on UMAP and spatial scatter plots per sample

9. differential gene expression

1. run `sc.tl.rank_genes_groups(groupby='leiden_harmony_0.6', method='wilcoxon')` for coarse clusters
2. repeat for `leiden_harmony_0.7`
3. export results as CSV files

10. save outputs

1. write `concatenated_sdata` zarr to `SDATA_OUT`
2. write processed AnnData to `ADATA_OUT` with `adata.write_h5ad()`

4. reclustering.ipynb

This script takes the h5ad output from the `combined_samples.ipynb` and annotates cell types from Leiden clusters, identifies spatially enriched populations (SEC cells, fibroblast progenitors, tdTomato+), performs sub-clustering, and transfers labels back to the main AnnData.

files:

- **notebook:** codes/reclustering.ipynb
- **input files:**
 - `combined_samples.h5ad` — processed AnnData from step 3; set path via `ADATA_PATH`
 - `concatenated_sdata/` — zarr SpatialData object from step 3; set path via `SDATA_PATH`
- **output files:**
 - `combined_samples_analyzed.h5ad` — AnnData with cell type labels and sub-cluster annotations; set path via `ADATA_OUT`

parameters:

parameter	value	description
FP recluster resolutions	0.4, 0.5, 0.6	resolutions for reclustering FP derivative cluster 0
tdTomato recluster resolutions	0.4, 0.5, 0.6	resolutions for reclustering tdTomato+ cells
SEC cell markers	ebf2+, sox9+, col2a1- (or col2a1+)	boolean raw-count co-expression masks for SEC cell identification
DEG method	Wilcoxon rank-sum	method used for all <code>rank_genes_groups</code> calls

process:

1. load data and fix zarr metadata

1. read `combined_samples.h5ad` and connect to `concatenated_sdata` zarr
2. repair any corrupt `.zarray` JSON metadata files in the `filtered_counts` and `norm_log` zarr layers by rewriting valid metadata with correct shape and chunk information

2. cell type annotation

1. map `leiden_harmony_0.7` cluster IDs to 17 manually assigned cell type labels based on marker gene expression
2. store labels in `adata.obs['cell_type']`

3. SEC cell identification

1. access raw counts from `adata.layers['filtered_counts']`
2. create boolean masks: `ebf2_pos = raw_counts[:, 'EBF2'] > 0`, `sox9_pos = raw_counts[:, 'SOX9'] > 0`, `col2a1_neg = raw_counts[:, 'COL2A1'] == 0`

3. identify SEC cells as `ebf2_pos & sox9_pos & col2a1_neg` (SEC-like) and `ebf2_pos & sox9_pos & col2a1_pos` (chondrocyte-like SEC)
4. run DEGs for SEC cells vs remaining `pdgfra+` cells

4. recluster FP derivatives

1. subset cells belonging to cluster 0 (FP derivatives/progenitors)
2. recompute PCA, Harmony correction, neighborhood graph, and UMAP on the subset
3. run Leiden at resolutions 0.4, 0.5, and 0.6
4. compute DEGs for each resolution and inspect marker genes

5. recluster tdTomato+ cells

1. subset cells where `adata.obs['tdTom1_custom'] > 0`
2. recompute embeddings and neighborhood graph
3. run Leiden at resolutions 0.4, 0.5, and 0.6
4. compute DEGs per resolution

6. transfer labels and save

1. transfer recluster labels from subsets back to the full AnnData via `obs` index alignment
2. write final annotated AnnData to `ADATA_OUT` with `adata.write_h5ad()`

5. tissues_one_canvas.ipynb

This is for visual purposes, where the h5ad outputs for both samples are stitched into one canvas. The output is another h5ad file, where all the samples can be visualized in one canvas. It extracts centroid spatial coordinates and polygon geometry from concatenated SpatialData, crops five tissue regions from two slides by coordinate bounds, applies geometric transformations (reflection, rotation), and stitches all tissues onto a shared canvas.

files:

- **input files:**
 - `combined_samples_analyzed.h5ad` — annotated AnnData from step 4; set path via `ADATA_PATH`
 - `concatenated_sdata/` — zarr SpatialData object; set path via `SDATA_PATH`
- **output files:**
 - `combined_samples_analyzed_one_canvas.h5ad` — AnnData with stitched canvas coordinates in `obs['spatial']`; set path via `ADATA_OUT`

parameters:

tissue	slide	x bounds	y bounds	transformation
E13.5	AB01	0–6990	950–5700	y-flip
E12.5 (1)	AB01	6300–11000	5700–9500	x-reflection
E12.5 (2)	AB02	2000–7000	3500–7500	y-flip
E11.5 (1)	AB02	9000–13500	1000–3500	+45° rotation
E11.5 (2)	AB02	8000–11000	6500–10500	-45° rotation

process:

1. extract centroids and geometry

1. load cell boundary polygons from `concatenated_sdata` (stored as shapely geometry or WKT strings)
2. compute centroid x/y for each cell and store in `adata.obsm['spatial']`
3. store full polygon geometry in `adata.obs['geometry']`

2. crop tissue regions

1. for each of the five tissues, apply coordinate bounds to subset cells from either slide AB01 or AB02
2. label each cell subset with the corresponding timepoint (E11.5, E12.5, or E13.5) in `adata.obs['timepoint']`

3. apply geometric transformations

1. y-flip: subtract y coordinates from the maximum y value to invert the axis
2. x-reflection: subtract x coordinates from the maximum x value to mirror horizontally
3. rotation: apply a 2D rotation matrix at $\pm 45^\circ$ using numpy matrix multiplication on the centroid coordinates

4. stitch onto shared canvas

1. define pixel offsets for each tissue region to place them side-by-side on a shared coordinate system
2. shift each tissue's centroid coordinates by its assigned offset
3. concatenate the five tissue subsets into a single AnnData, preserving all obs columns and layers

5. save

1. write the stitched AnnData to `ADATA_OUT` with `adata.write_h5ad()`

6. plot_with_grid.py

This script is just a util script to helping with cropping the samples visually.

files:

- **input:** an AnnData with `obsm['spatial']` coordinates and a SpatialData object with cell boundaries

process:

`plot_samples_with_grid(adata, samples, group, concatenated_sdata)`

1. accepts a list of sample names and a grouping obs column (e.g. 'leiden_harmony_0.7')
2. for each sample, plots cell boundaries colored by the group variable
3. overlays a major coordinate grid (500-unit intervals) and a minor coordinate grid (100-unit intervals) to enable precise region identification
4. useful for determining `array_col` / `array_row` bounds before cropping

`crop_with_grids(adata)`

1. creates a scatter plot of cells using `adata.obsm['spatial']` x/y coordinates
 2. colors points by Leiden cluster assignment
 3. overlays a coordinate grid at configurable intervals for visual reference
-

7. trajectory_analysis_original.ipynb

This is just an exploratory notebook for pdgfra+ and tdTomato+ trajectory analysis: loads annotated data, subsets mesenchymal populations, recalculates embeddings, and applies sub-clustering with DEGs before running trajectory methods.

files:

- **notebook:** `codes/trajectory_analysis_original.ipynb`
- **input files:**
 - `combined_samples_analyzed_one_canvas_labeled.h5ad` — canvas-stitched, labeled AnnData; set path via `ADATA_PATH`
- **output files:**
 - intermediate subsets written to disk as exploration checkpoints; paths set by user in notebook

process:

1. load and visualize

1. read `combined_samples_analyzed_one_canvas_labeled.h5ad`
2. parse WKT geometry strings back to shapely objects using `shapely.wkt.loads()`
3. visualize cell type labels and timepoint annotations spatially

2. subset pdgfra+ cells

1. subset cells where `cell_type` matches pdgfra+ annotations
2. recompute PCA (50 components), Harmony batch correction (`key="sample"`), neighborhood graph, and UMAP on the subset

3. subset mesenchymal clusters

1. further subset to mesenchymal cell types: Cartilage, LPM, Tendon, Dermal fibroblast, Fibroblasts including BA progenitors, Chondrocytes, Dermal bone
2. recompute PCA, Harmony, neighbors, and UMAP
3. run Leiden clustering at resolutions 0.6, 0.7, and 0.8
4. compute DEGs for each resolution via `sc.tl.rank_genes_groups(method='wilcoxon')`

4. subset tdTomato+ cells

1. subset cells where `tdTom1_custom > 0`
2. recompute PCA, Harmony, neighborhood graph, and UMAP
3. visualize with cell type and timepoint color keys

8. pdgfra_analysis.ipynb

This script is for trajectory analysis fo pdgfra+ cells using Cellrank and Moscot.

files:

- **input files:**
 - `pdgfra_gpcca.h5ad` — checkpoint h5ad with stored transition matrix for pdgfra+ cells; set path via `PDGFRA_ADATA_PATH`
 - `tdtom_gpcca.h5ad` — checkpoint h5ad for tdTomato+ cells; set path via `TDTOM_ADATA_PATH`
- **output files:**
 - updated h5ad files and CSV lineage driver tables; set paths via `PDGFRA_OUT` and `TDTOM_OUT`

parameters:

parameter	value	description
CFLARE k	10	number of clusters for kmeans terminal state prediction
GPCCA Schur method	'krylov'	eigendecomposition method
GPCCA n_states (pdgfra)	10, 12	number of macrostates tested
moscot epsilon	1e-3	entropic regularization for optimal transport
moscot tau_a	0.95	unbalancedness parameter (marginal penalty)
GPCCA n_states (tdtomato)	6	number of macrostates for tdtomato analysis

process:

1. pdgfra+ CFLARE analysis

1. load `pdgfra_gpcca.h5ad`
2. rebuild `RealTimeKernel` from stored `T_fwd` matrix using `RealTimeKernel.from_adata(adata, key="T_fwd")`
3. initialize CFLARE estimator: `g = cr.estimators.CFLARE(rtk)`
4. fit with `g.fit(k=10)` and predict terminal states with `method='kmeans'` and `n_clusters_kmeans=10`
5. compute fate probabilities towards each terminal state

2. pdgfra+ GPCCA analysis

1. initialize GPCCA estimator: `g = cr.estimators.GPCCA(rtk)`
2. compute Schur decomposition: `g.compute_schur(method='krylov')`
3. compute macrostates for `n_states=10` and `n_states=12`; select the best decomposition
4. predict terminal and initial states
5. compute fate probabilities with `g.compute_fate_probabilities()`

3. tdtomato GPCCA analysis

1. load `tdtom_gpcca.h5ad`
2. run fresh moscot `TemporalProblem` with `epsilon=1e-3`, `tau_a=0.95`
3. build `RealTimeKernel` from the moscot solution
4. initialize and fit GPCCA estimator with `n_states=6`
5. compute fate probabilities

9. tdtomato__analysis.ipynb

This is for analyzing trajectory of tdtomato+ cells. This script also contains reclustering of myotome/FP sub-populations, runs moscot

optimal transport for developmental flow analysis, and saves the final labeled AnnData.

files:

- **input files:**
 - `tdtomato_labeled.h5ad` — labeled AnnData for tdTomato+ cells (35,522 cells); set path via `TDTOM_PATH`
- **output files:**
 - `tdtomato_labeled.h5ad` — updated in place with refined annotations; path set via `TDTOM_OUT`

parameters:

parameter	value	description
Mesenchyme recluster resolution	0.4	Leiden resolution for mesenchyme including BA subset
Myotome/FP recluster resolutions	0.2, 0.3, 0.4	Leiden resolutions for E11.5 Myotome and FP subset
moscot epsilon	1e-3	entropic regularization
moscot tau_a	0.95	unbalancedness parameter

process:

1. subset and recluster mesenchyme including BA

1. subset cells where `cell_type == "Mesenchyme including BA"` (7,270 cells)
2. recompute PCA, Harmony, neighborhood graph, and UMAP
3. run Leiden at resolution 0.4
4. annotate 10 sub-types: Brown adipocytes, Tendon, CT-like fibroblasts, and others based on marker genes
5. transfer sub-type labels back to the parent AnnData by obs index

2. subset and recluster E11.5 Myotome and FP

1. subset cells from the E11.5 Myotome and FP cluster
2. recompute embeddings
3. run Leiden at resolutions 0.2, 0.3, and 0.4
4. assign anatomical labels (e.g. myoblasts, floor plate derivatives) based on DEGs

3. moscot developmental flow

1. initialize `TemporalProblem` with `epsilon=1e-3`, `tau_a=0.95`

2. solve optimal transport between consecutive timepoints (E11.5 $\rightarrow$ E12.5 $\rightarrow$ E13.5)
3. plot probability mass flow per cluster to visualize lineage trajectories

4. save

1. write updated AnnData to TDTOM_OUT with `adata.write_h5ad()`

10. trajectory.py

HPC script that runs the full Cellrank analysis pipeline for both pdgfra+ and tdtomato+ cell populations, including moscot optimal transport, RealTimeKernel construction, GPCCA macrostate analysis, and fate probability computation. This is not used anymore since Cellrank was determined not useful for trajectory analysis.

files:

- **input files:**
 - labeled AnnData file for pdgfra+ and tdtomato+ cells; set `DATA_PATH` at the top of the script to the HPC data directory
- **output files:**
 - `{label}_tmat.h5ad` — AnnData with stored transition matrix; set `OUT_DIR_PDGFR` and `OUT_DIR_TDTOM`
 - `{label}_gpcca.h5ad` — AnnData with fate probabilities after GPCCA; same output directories

parameters:

parameter	value	description
JAX memory fraction	0.80	fraction of GPU memory allocated to JAX (<code>XLA_PYTHON_CLIENT_MEM_FRACTION</code>)
CUDA allocator	async	
OT rank	500	low-rank approximation for CPU-based OT solve to avoid GPU OOM
moscot epsilon	1e-3	entropic regularization
moscot tau_a	0.95	unbalancedness parameter
GPCCA	10–12	number of macrostates
n_states		

process:

1. environment setup

1. set JAX/XLA environment variables for GPU memory management before importing JAX or CellRank
2. configure async CUDA allocator and 80% memory fraction

2. run_trajectory(adata, label, out_dir) function

1. compute marginal scores for each timepoint (E11.5, E12.5, E13.5) for unbalancedness penalties
2. initialize `TemporalProblem` and call `.prepare()` and `.solve(epsilon=1e-3, tau_a=0.95, rank=500)`
3. construct `RealTimeKernel` from the moscot solution
4. checkpoint: drop `coarse_fwd` from `adata.uns` to avoid `AnnData` write errors, then save `{label}_tmat.h5ad`
5. initialize `CFLARE` estimator, fit, and predict terminal states
6. initialize `GPCCA` estimator, compute Schur decomposition, compute macrostates and fate probabilities
7. checkpoint: save `{label}_gpcca.h5ad`

3. run for both populations

1. load and subset `pdgfra+` cells, call `run_trajectory(pdgfra_adata, "pdgfra", OUT_DIR_PDGFRA)`
 2. load and subset `tdtomato+` cells, call `run_trajectory(tdtom_adata, "tdtom", OUT_DIR_TDTOM)`
-

12. trajectory_pdgfra_only.py

same pipeline as `trajectory.py` but runs only the `pdgfra+` cell population. use when re-running or debugging `pdgfra+` analysis independently of `tdtomato+`. This is not used anymore.

files:

- **script:** `codes/trajectory_pdgfra_only.py`
- **input files:** same as `trajectory.py` — set `DATA_PATH` to the HPC data directory
- **output files:** same checkpoint `h5ad` files in `OUT_DIR_PDGFRA`

process:

follows the same steps as `trajectory.py` (see above), executing only the `pdgfra+` branch of `run_trajectory()`.

13. resume_lineage_drivers.py

resumes lineage driver computation from a saved GPCCA checkpoint, reattaching stored fate probabilities and computing lineage driver genes for a selected terminal lineage. This is not used anymore.

files:

- **script:** codes/resume_lineage_drivers.py
- **input files:**
 - pdgfra_gpcca.h5ad — GPCCA checkpoint; set path via PDGFRA_ADATA_PATH in the script
- **output files:**
 - pdgfra_lineage_drivers.csv — ranked lineage driver genes; set path via DRIVERS_OUT

process:

1. load checkpoint and rebuild kernel

1. read pdgfra_gpcca.h5ad
2. rebuild RealTimeKernel with RealTimeKernel.from_adata(adata, key="T_fwd")

2. reattach fate probabilities

1. initialize CFLARE estimator
2. reattach stored fate probability matrix from adata.obsm (stored by a prior trajectory.py run)

3. compute lineage drivers

1. call g.compute_lineage_drivers(lineages=["Fibroblasts including BA progenitors"])
2. this correlates gene expression with fate probability to rank driver genes

4. save

1. export the ranked driver gene table to DRIVERS_OUT as a CSV file
-

14. run_trajectory.sh

SLURM job submission script for running trajectory.py on the insomnia001 HPC cluster. Just a util script.

files:

- **script:** `codes/run_trajectory.sh`
- **requirements:** SLURM scheduler, conda environment `sc_cellrank` at the HPC conda path, GPU node with at least 240 GB RAM

SLURM parameters:

parameter	value	description
<code>--account</code>	<code>pmg</code>	SLURM account/partition
<code>--time</code>	<code>10:00:00</code>	maximum job wall time (10 hours)
<code>--mem</code>	<code>240G</code>	RAM allocation
<code>--cpus-per-task</code>	<code>8</code>	CPU core allocation
<code>--gres</code>	<code>gpu:1</code>	request 1 GPU
<code>--exclude</code>	<code>ins092,ins082</code>	nodes excluded due to known issues

process:

1. activate conda environment

1. source the HPC conda initialization script (update path to match your HPC setup)
2. activate the `sc_cellrank` environment: `conda activate sc_cellrank`

2. run trajectory script

1. execute `python trajectory.py` within the activated environment
2. output and error logs are captured by SLURM to files set via `--output` and `--error` flags (set these paths before submitting)

3. submit job

1. from the HPC login node, run `sbatch run_trajectory.sh`
2. monitor with `squeue -u <username>` or `sacct -j <jobid>`